

)

5. The responder SHALL encode the following items as a **sigma-2-tbsdata** with an **anonymous tag**:
 - a. **responderNOC** as a **matter-certificate**
 - b. **responderICAC** (if present) as a **matter-certificate**
 - c. **ResponderEphKeyPair.publicKey**
 - d. **Msg1.initiatorEphPubKey**
6. The responder SHALL **generate a signature**:

```

TBSDData2Signature = Crypto_Sign(
    message = sigma-2-tbsdata,
    privateKey = ResponderNOKeyPair.privateKey
)

```

7. The responder SHALL encode the following items as a **sigma-2-tbedata**, where the encoding of **responderNOC** and **responderICAC** items SHALL be byte-for-byte identical to the encoding in **sigma-2-tbsdata**:
 - a. **responderNOC** as a **matter-certificate**. This encoding SHALL be byte-for-byte identical to the encoding in **sigma-2-tbsdata**.
 - b. **responderICAC** (if present) as a **matter-certificate**. This encoding SHALL be byte-for-byte identical to the encoding in **sigma-2-tbsdata**.
 - c. **TBSDData2Signature**
 - d. **ResumptionID**
8. The responder SHALL **generate a random number** **Random = Crypto_DRBG(len = 32 * 8)**.
9. The responder SHALL **generate the S2K key** using **Random** as Responder Random and **ResponderEphKeyPair.publicKey** as Responder Ephemeral Public Key.
10. The responder SHALL **generate the encrypted and integrity protected data**:

```

byte TBEDData2_A[] = {}
byte TBEDData2_Nonce[13] = /* "NCASE_Sigma2N" */
    {0x4e, 0x43, 0x41, 0x53, 0x45, 0x5f, 0x53, 0x69,
     0x67, 0x6d, 0x61, 0x32, 0x4e}

TBEDData2Encrypted = Crypto_AEAD_GenerateEncrypt(
    K = S2K,
    P = TBEDData2,
    A = TBEDData2_A,
    N = TBEDData2_Nonce
)

```